PRACTICE PROJECT SOLUTIONS

This appendix provides the solutions to the practice projects in each chapter. Digital versions are available on the book's website at **https://www.nostarch.com/impracticalpython/**.

Chapter 1: Silly Name Generator

Pig Latin

pig_Latin_practice.py

```
"""Turn a word into its Pig Latin equivalent."""
import sys

VOWELS = 'aeiouy'

while True:
    word = input("Type a word and get its Pig Latin translation: ")

    if word[0] in VOWELS:
        pig_Latin = word + 'way'
    else:
        pig_Latin = word[1:] + word[0] + 'ay'
    print()
    print("{}".format(pig_Latin), file=sys.stderr)
```

```
    try_again = input("\n\nTry again? (Press Enter else n to stop)\n ")
    if try_again.lower() == "n":
        sys.exit()
```

Poor Man's Bar Chart

EATOIN_practice.py

```
"""Map letters from string into dictionary & print bar chart of frequency."""
import sys
import pprint
from collections import defaultdict

# Note: text should be a short phrase for bars to fit in IDLE window
text = 'Like the castle in its corner in a medieval game, I foresee terrible \
trouble and I stay here just the same.'

ALPHABET = 'abcdefghijklmnopqrstuvwxyz'

# defaultdict module lets you build dictionary keys on the fly!
mapped = defaultdict(list)
for character in text:
    character = character.lower()
    if character in ALPHABET:
        mapped[character].append(character)

# pprint lets you print stacked output
print("\nYou may need to stretch console window if text wrapping occurs.\n")
print("text = ", end='')
print("{}\n".format(text), file=sys.stderr)
pprint.pprint(mapped, width=110)
```

Chapter 2: Finding Palingram Spells

Dictionary Cleanup

dictionary_cleanup_practice.py

```
"""Remove single-letter words from list if not 'a' or 'i'."""
word_list = ['a', 'nurses', 'i', 'stack', 'b', 'c', 'cat']
word_list_clean = []

permissible = ('a', 'i')

for word in word_list:
    if len(word) > 1:
        word_list_clean.append(word)
    elif len(word) == 1 and word in permissible:
        word_list_clean.append(word)
    else:
        continue

print("{}".format(word_list_clean))
```

Chapter 3: Solving Anagrams

Finding Digrams

count_digrams_practice.py

```
"""Generate letter pairs in Voldemort & find their frequency in a dictionary.

Requires load_dictionary.py module to load an English dictionary file.

"""
import re
from collections import defaultdict
from itertools import permutations
import load_dictionary

word_list = load_dictionary.load('2of4brif.txt')

name = 'Voldemort'  #(tmvoordle)
name = name.lower()
```

```
# generate unique letter pairs from name
digrams = set()
perms = {''.join(i) for i in permutations(name)}
for perm in perms:
    for i in range(0, len(perm) - 1):
        digrams.add(perm[i] + perm[i + 1])
print(*sorted(digrams), sep='\n')
print("\nNumber of digrams = {}\n".format(len(digrams)))

# use regular expressions to find repeating digrams in a word
mapped = defaultdict(int)
for word in word_list:
    word = word.lower()
    for digram in digrams:
        for m in re.finditer(digram, word):
            mapped[digram] += 1

print("digram frequency count:")
count = 0
for k in sorted(mapped):
    print("{} {}".format(k, mapped[k]))
```

Chapter 4: Decoding American Civil War Ciphers

Hacking Lincoln

Code word	Plaintext
WAYLAND	captured
NEPTUNE	Richmond

Plaintext: correspondents of the Tribune captured at Richmond please ascertain why they are detained and get them off if you can this fills it up

Identifying Cipher Types

identify_cipher_type_practice.py

```
"""Load ciphertext & use fraction of ETAOIN present to classify cipher type."""
import sys
from collections import Counter

# set arbitrary cutoff fraction of 6 most common letters in English
# ciphertext with target fraction or greater = transposition cipher
CUTOFF = 0.5

# load ciphertext
def load(filename):
    """Open text file and return list."""
    with open(filename) as f:
        return f.read().strip()

try:
    ciphertext = load('cipher_a.txt')
except IOError as e:
    print("{}. Terminating program.".format(e),
          file=sys.stderr)
    sys.exit(1)

# count 6 most common letters in ciphertext
six_most_frequent = Counter(ciphertext.lower()).most_common(6)
print("\nSix most-frequently-used letters in English = ETAOIN")
print('\nSix most frequent letters in ciphertext =')
print(*six_most_frequent, sep='\n')

# convert list of tuples to set of letters for comparison
cipher_top_6 = {i[0] for i in six_most_frequent}

TARGET = 'etaoin'
count = 0
for letter in TARGET:
    if letter in cipher_top_6:
        count += 1

if count/len(TARGET) >= CUTOFF:
```

```
    print("\nThis ciphertext most-likely produced by a TRANSPOSITION cipher")
else:
    print("This ciphertext most-likely produced by a SUBSTITUTION cipher")
```

Storing a Key as a Dictionary

key_dictionary_practice.py

```
"""Input cipher key string, get user input on route direction as dict value."""
col_order = """1 3 4 2"""
key = dict()
cols = [int(i) for i in col_order.split()]
for col in cols:
    while True:
        key[col] = input("Direction to read Column {} (u = up, d = down): "
                         .format(col).lower())
        if key[col] == 'u' or key[col] == 'd':
            break
        else:
            print("Input should be 'u' or 'd'")

    print("{}, {}".format(col, key[col]))
```

Automating Possible Keys

permutations_practice.py

```
"""For a total number of columns, find all unique column arrangements.

Builds a list of lists containing all possible unique arrangements of
individual column numbers, including negative values for route direction
(read up column vs. down).

Input:
-total number of columns

Returns:
```

```
-list of lists of unique column orders, including negative values for
route cipher encryption direction

"""
import math
from itertools import permutations, product

#------BEGIN INPUT---------------------------------------------------------

# Input total number of columns:
num_cols = 4

#------DO NOT EDIT BELOW THIS LINE-----------------------------------------


# generate listing of individual column numbers
columns = [x for x in range(1, num_cols+1)]
print("columns = {}".format(columns))

# build list of lists of column number combinations
# itertools product computes the Cartesian product of input iterables
def perms(columns):
    """Take number of columns integer & generate pos & neg permutations."""
    results = []
    for perm in permutations(columns):
        for signs in product([-1, 1], repeat=len(columns)):
            results.append([i*sign for i, sign in zip(perm, signs)])
    return results

col_combos = perms(columns)
print(*col_combos, sep="\n")  # comment-out for num_cols > 4!
print("Factorial of num_cols without negatives = {}"
      .format(math.factorial(num_cols)))
print("Number of column combinations = {}".format(len(col_combos)))
```

Route Transposition Cipher: Brute-Force Attack

This practice project uses two programs. The second, *perms.py*, is used as a module in the first program, *route_cipher_hacker.py*. It was built from the *permutations_practice.py* program previously described in "**Automating Possible Keys**" on **page 371**.

route_cipher_hacker.py

route_cipher_hacker.py

```
"""Brute-force hack a Union route cipher (route_cipher_hacker.py).

Designed for whole-word transposition ciphers with variable rows & columns.
Assumes encryption began at either top or bottom of a column.
Possible keys auto-generated based on number of columns & rows input.
Key indicates the order to read columns and the direction to traverse.
Negative column numbers mean start at bottom and read up.
Positive column numbers means start at top & read down.

Example below is for 4x4 matrix with key -1 2 -3 4.
Note "0" is not allowed.
Arrows show encryption route; for negative key values read UP.

  1   2   3   4
___ ___ ___ ___
| ^ | | | ^ | | | MESSAGE IS WRITTEN
|_|_|_v_|_|_|_v_|
| ^ | | | ^ | | | ACROSS EACH ROW
|_|_|_v_|_|_|_v_|
| ^ | | | ^ | | | IN THIS MANNER
|_|_|_v_|_|_|_v_|
| ^ | | | ^ | | | LAST ROW IS FILLED WITH DUMMY WORDS
|_|_|_v_|_|_|_v_|
START        END

Required inputs - a text message, # of columns, # of rows, key string
Requires custom-made "perms" module to generate keys
Prints off key used and translated plaintext
"""
```

```
import sys
import perms

#============================================================================
# USER INPUT:

# the string to be decrypted (type or paste between triple-quotes):
ciphertext = """REST TRANSPORT YOU GODWIN VILLAGE ROANOKE WITH ARE YOUR IS JUST
SUPPLIES FREE SNOW HEADING TO GONE TO SOUTH FILLER
"""

# the number of columns believed to be in the transposition matrix:
COLS = 4

# the number of rows believed to be in the transposition matrix:
ROWS = 5

# END OF USER INPUT - DO NOT EDIT BELOW THIS LINE!
#============================================================================



def main():
    """Turn ciphertext into list, call validation & decryption functions."""
    cipherlist = list(ciphertext.split())
    validate_col_row(cipherlist)
    decrypt(cipherlist)

def validate_col_row(cipherlist):
    """Check that input columns & rows are valid vs. message length."""
    factors = []
    len_cipher = len(cipherlist)
    for i in range(2, len_cipher):  # range excludes 1-column ciphers
        if len_cipher % i == 0:
            factors.append(i)
    print("\nLength of cipher = {}".format(len_cipher))
    print("Acceptable column/row values include: {}".format(factors))
    print()
```

```
    if ROWS * COLS != len_cipher:
        print("\nError - Input columns & rows not factors of length "
              "of cipher. Terminating program.", file=sys.stderr)
        sys.exit(1)

def decrypt(cipherlist):
    """Turn columns into items in list of lists & decrypt ciphertext."""
    col_combos = perms.perms(COLS)
    for key in col_combos:
        translation_matrix = [None] * COLS
        plaintext = ''
        start = 0
        stop = ROWS
        for k in key:
            if k < 0: # reading bottom-to-top of column
                col_items = cipherlist[start:stop]
            elif k > 0: # reading top-to-bottom of columnn
                col_items = list((reversed(cipherlist[start:stop])))
            translation_matrix[abs(k) - 1] = col_items
            start += ROWS
            stop += ROWS
        # loop through nested lists popping off last item to a new list:
        for i in range(ROWS):
            for matrix_col in translation_matrix:
                word = str(matrix_col.pop())
                plaintext += word + ' '
        print("\nusing key = {}".format(key))
        print("translated = {}".format(plaintext))
    print("\nnumber of keys = {}".format(len(col_combos)))

if __name__ == '__main__':
    main()
```

---

perms.py

perms.py

```
    """For a total number of columns, find all unique column arrangements.

    Builds a list of lists containing all possible unique arrangements of
    individual column numbers including negative values for route direction

    Input:
    -total number of columns

    Returns:
    -list of lists of unique column orders including negative values for
    route cipher encryption direction

    """
    from itertools import permutations, product

    # build list of lists of column number combinations
    # itertools product computes the Cartesian product of input iterables
    def perms(num_cols):
        """Take number of columns integer & generate pos & neg permutations."""
        results = []
        columns = [x for x in range(1, num_cols+1)]
        for perm in permutations(columns):
            for signs in product([-1, 1], repeat=len(columns)):
                results.append([i*sign for i, sign in zip(perm, signs)])
        return results
```

Chapter 5: Encoding English Civil War Ciphers

Saving Mary

save_Mary_practice.py

```
    """Hide a null cipher within a list of names using a variable pattern."""
    import load_dictionary

    # write a short message and use no punctuation or numbers!
    message = "Give your word and we rise"
```

```
message = "".join(message.split())

# open name file
names = load_dictionary.load('supporters.txt')

name_list = []

# start list with null word not used in cipher
name_list.append(names[0])

# add letter of null cipher to 2nd letter of name, then 3rd, then repeat
count = 1
for letter in message:
    for name in names:
        if len(name) > 2 and name not in name_list:
            if count % 2 == 0 and name[2].lower() == letter.lower():
                name_list.append(name)
                count += 1
                break
            elif count % 2 != 0 and name[1].lower() == letter.lower():
                name_list.append(name)
                count += 1
                break

# add two null words early in message to throw off cryptanalysts
name_list.insert(3, 'Stuart')
name_list.insert(6, 'Jacob')

# display cover letter and list with null cipher
print("""
Your Royal Highness: \n
It is with the greatest pleasure I present the list of noble families who
have undertaken to support your cause and petition the usurper for the
release of your Majesty from the current tragical circumstances.
""")

print(*name_list, sep='\n')
```

The Colchester Catch

colchester_practice.py

```
"""Solve a null cipher based on every nth letter in every nth word."""
import sys

def load_text(file):
    """Load a text file as a string."""
    with open(file) as f:
        return f.read().strip()

# load & process message:
filename = input("\nEnter full filename for message to translate: ")
try:
    loaded_message = load_text(filename)
except IOError as e:
    print("{}. Terminating program.".format(e), file=sys.stderr)
    sys.exit(1)

# check loaded message & # of lines
print("\nORIGINAL MESSAGE = {}\n".format(loaded_message))

# convert message to list and get length
message = loaded_message.split()
end = len(message)

# get user input on interval to check
increment = int(input("Input max word & letter position to \
                      check (e.g., every 1 of 1, 2 of 2, etc.): "))
print()

# find letters at designated intervals
for i in range(1, increment + 1):
    print("\nUsing increment letter {} of word {}".format(i, i))
    print()
    count = i - 1
    location = i - 1
```

```
    for index, word in enumerate(message):
        if index == count:
            if location < len(word):
                print("letter = {}".format(word[location]))
                count += i
            else:
                print("Interval doesn't work", file=sys.stderr)
```

Chapter 6: Writing in Invisible Ink

Checking the Number of Blank Lines

elementary_ink_practice.py

```
"""Add code to check blank lines in fake message vs lines in real message."""
import sys
import docx
from docx.shared import RGBColor, Pt

# get text from fake message & make each line a list item
fake_text = docx.Document('fakeMessage.docx')
fake_list = []
for paragraph in fake_text.paragraphs:
    fake_list.append(paragraph.text)

# get text from real message & make each line a list item
real_text = docx.Document('realMessageChallenge.docx')
real_list = []
for paragraph in real_text.paragraphs:
    if len(paragraph.text) != 0:  # remove blank lines
        real_list.append(paragraph.text)

# define function to check available hiding space:
def line_limit(fake, real):
    """Compare number of blank lines in fake vs lines in real and
    warn user if there are not enough blanks to hold real message.

    NOTE:  need to import 'sys'
```

```
    """
    num_blanks = 0
    num_real = 0
    for line in fake:
        if line == '':
            num_blanks += 1
    num_real = len(real)
    diff = num_real - num_blanks
    print("\nNumber of blank lines in fake message = {}".format(num_blanks))
    print("Number of lines in real message = {}\n".format(num_real))
    if num_real > num_blanks:
        print("Fake message needs {} more blank lines."
              .format(diff), file=sys.stderr)
        sys.exit()

line_limit(fake_list, real_list)

# load template that sets style, font, margins, etc.
doc = docx.Document('template.docx')

# add letterhead
doc.add_heading('Morland Holmes', 0)
subtitle = doc.add_heading('Global Consulting & Negotiations', 1)
subtitle.alignment = 1
doc.add_heading('', 1)
doc.add_paragraph('December 17, 2015')
doc.add_paragraph('')

def set_spacing(paragraph):
    """Use docx to set line spacing between paragraphs."""
    paragraph_format = paragraph.paragraph_format
    paragraph_format.space_before = Pt(0)
    paragraph_format.space_after = Pt(0)

length_real = len(real_list)
count_real = 0  # index of current line in real (hidden) message
```

```
# interleave real and fake message lines
for line in fake_list:
    if count_real < length_real and line == "":
        paragraph = doc.add_paragraph(real_list[count_real])
        paragraph_index = len(doc.paragraphs) - 1

        # set real message color to white
        run = doc.paragraphs[paragraph_index].runs[0]
        font = run.font
        font.color.rgb = RGBColor(255, 255, 255)  # make it red to test
        count_real += 1

    else:
        paragraph = doc.add_paragraph(line)

    set_spacing(paragraph)

doc.save('ciphertext_message_letterhead.docx')

print("Done"))
```

Chapter 8: Counting Syllables for Haiku Poetry

Syllable Counter vs. Dictionary File

test_count_syllables_w_dict.py

```
"""Load a dictionary file, pick random words, run syllable-counting module."""
import sys
import random
from count_syllables import count_syllables

def load(file):
    """Open a text file & return list of lowercase strings."""
    with open(file) as in_file:
        loaded_txt = in_file.read().strip().split('\n')
        loaded_txt = [x.lower() for x in loaded_txt]
        return loaded_txt
```

```
try:
    word_list = load('2of4brif.txt')
except IOError as e:
    print("{}\nError opening file. Terminating program.".format(e),
          file=sys.stderr)
    sys.exit(1)

test_data = []
num_words = 100
test_data.extend(random.sample(word_list, num_words))

for word in test_data:
    try:
        num_syllables = count_syllables(word)
        print(word, num_syllables, end='\n')
    except KeyError:
        print(word, end='')
        print(" not found", file=sys.stderr)
```

Chapter 10: Are We Alone? Exploring the Fermi Paradox

A Galaxy Far, Far Away

galaxy_practice.py

```
"""Use spiral formula to build galaxy display."""
import math
from random import randint
import tkinter

root = tkinter.Tk()
root.title("Galaxy BR549")
c = tkinter.Canvas(root, width=1000, height=800, bg='black')
c.grid()
c.configure(scrollregion=(-500, -400, 500, 400))
oval_size = 0

# build spiral arms
```

```
num_spiral_stars = 500
angle = 3.5
core_diameter = 120
spiral_stars = []
for i in range(num_spiral_stars):
    theta = i * angle
    r = math.sqrt(i) / math.sqrt(num_spiral_stars)
    spiral_stars.append((r * math.cos(theta), r * math.sin(theta)))
for x, y in spiral_stars:
    x = x * 350 + randint(-5, 3)
    y = y * 350 + randint(-5, 3)
    oval_size = randint(1, 3)
    c.create_oval(x-oval_size, y-oval_size, x+oval_size, y+oval_size,
                  fill='white', outline='')

# build wisps
wisps = []
for i in range(2000):
    theta = i * angle
    # divide by num_spiral_stars for better dust lanes
    r = math.sqrt(i) / math.sqrt(num_spiral_stars)
    spiral_stars.append((r * math.cos(theta), r * math.sin(theta)))
for x, y in spiral_stars:
    x = x * 330 + randint(-15, 10)
    y = y * 330 + randint(-15, 10)
    h = math.sqrt(x**2 + y**2)
    if h < 350:
        wisps.append((x, y))
        c.create_oval(x-1, y-1, x+1, y+1, fill='white', outline='')

# build galactic core
core = []
for i in range(900):
    x = randint(-core_diameter, core_diameter)
    y = randint(-core_diameter, core_diameter)
    h = math.sqrt(x**2 + y**2)
    if h < core_diameter - 70:
        core.append((x, y))
```

```
        oval_size = randint(2, 4)
        c.create_oval(x-oval_size, y-oval_size, x+oval_size, y+oval_size,
                      fill='white', outline='')
    elif h < core_diameter:
        core.append((x, y))
        oval_size = randint(0, 2)
        c.create_oval(x-oval_size, y-oval_size, x+oval_size, y+oval_size,
                      fill='white', outline='')

root.mainloop()
```

Building a Galactic Empire

empire_practice.py

```
"""Build 2-D model of galaxy, post expansion rings for galactic empire."""
import tkinter as tk
import time
from random import randint, uniform, random
import math

#=============================================================================
# MAIN INPUT

# location of galactic empire homeworld on map:
HOMEWORLD_LOC = (0, 0)

# maximum number of years to simulate:
MAX_YEARS = 10000000

# average expansion velocity as fraction of speed of light:
SPEED = 0.005

# scale units
UNIT = 200

#=====================================================================
```

```
# set up display canvas
root = tk.Tk()
root.title("Milky Way galaxy")
c = tk.Canvas(root, width=1000, height=800, bg='black')
c.grid()
c.configure(scrollregion=(-500, -400, 500, 400))

# actual Milky Way dimensions (light-years)
DISC_RADIUS = 50000

disc_radius_scaled = round(DISC_RADIUS/UNIT)

def polar_coordinates():
    """Generate uniform random x,y point within a disc for 2-D display."""
    r = random()
    theta = uniform(0, 2 * math.pi)
    x = round(math.sqrt(r) * math.cos(theta) * disc_radius_scaled)
    y = round(math.sqrt(r) * math.sin(theta) * disc_radius_scaled)
    return x, y

def spirals(b, r, rot_fac, fuz_fac, arm):
    """Build spiral arms for tkinter display using Logarithmic spiral formula.

    b = arbitrary constant in logarithmic spiral equation
    r = scaled galactic disc radius
    rot_fac = rotation factor
    fuz_fac = random shift in star position in arm, applied to 'fuzz' variable
    arm = spiral arm (0 = main arm, 1 = trailing stars)
    """
    spiral_stars = []
    fuzz = int(0.030 * abs(r))  # randomly shift star locations
    theta_max_degrees = 520
    for i in range(theta_max_degrees):  # range(0, 700, 2) for no black hole
        theta = math.radians(i)
        x = r * math.exp(b*theta) * math.cos(theta + math.pi * rot_fac)\
            + randint(-fuzz, fuzz) * fuz_fac
        y = r * math.exp(b*theta) * math.sin(theta + math.pi * rot_fac)\
```

```
            + randint(-fuzz, fuzz) * fuz_fac
        spiral_stars.append((x, y))
    for x, y in spiral_stars:
        if arm == 0 and int(x % 2) == 0:
            c.create_oval(x-2, y-2, x+2, y+2, fill='white', outline='')
        elif arm == 0 and int(x % 2) != 0:
            c.create_oval(x-1, y-1, x+1, y+1, fill='white', outline='')
        elif arm == 1:
            c.create_oval(x, y, x, y, fill='white', outline='')

def star_haze(scalar):
    """Randomly distribute faint tkinter stars in galactic disc.
    disc_radius_scaled = galactic disc radius scaled to radio bubble diameter
    scalar = multiplier to vary number of stars posted
    """
    for i in range(0, disc_radius_scaled * scalar):
        x, y = polar_coordinates()
        c.create_text(x, y, fill='white', font=('Helvetica', '7'), text='.')

def model_expansion():
    """Model empire expansion from homeworld with concentric rings."""
    r = 0 # radius from homeworld
    text_y_loc = -290
    x, y = HOMEWORLD_LOC
    c.create_oval(x-5, y-5, x+5, y+5, fill='red')
    increment = round(MAX_YEARS / 10)# year interval to post circles
    c.create_text(-475, -350, anchor='w', fill='red', text='Increment = {:,}'
                  .format(increment))
    c.create_text(-475, -325, anchor='w', fill='red',
                  text='Velocity as fraction of Light = {:,}'.format(SPEED))

    for years in range(increment, MAX_YEARS + 1, increment):
        time.sleep(0.5) # delay before posting new expansion circle
        traveled = SPEED * increment / UNIT
        r = r + traveled
        c.create_oval(x-r, y-r, x+r, y+r, fill='', outline='red', width='2')
        c.create_text(-475, text_y_loc, anchor='w', fill='red',
                      text='Years = {:,}'.format(years))
```

```
        text_y_loc += 20
        # update canvas for new circle; no longer need mainloop()
        c.update_idletasks()
        c.update()

def main():
    """Generate galaxy display, model empire expansion, run mainloop."""
    spirals(b=-0.3, r=disc_radius_scaled, rot_fac=2, fuz_fac=1.5, arm=0)
    spirals(b=-0.3, r=disc_radius_scaled, rot_fac=1.91, fuz_fac=1.5, arm=1)
    spirals(b=-0.3, r=-disc_radius_scaled, rot_fac=2, fuz_fac=1.5, arm=0)
    spirals(b=-0.3, r=-disc_radius_scaled, rot_fac=-2.09, fuz_fac=1.5, arm=1)
    spirals(b=-0.3, r=-disc_radius_scaled, rot_fac=0.5, fuz_fac=1.5, arm=0)
    spirals(b=-0.3, r=-disc_radius_scaled, rot_fac=0.4, fuz_fac=1.5, arm=1)
    spirals(b=-0.3, r=-disc_radius_scaled, rot_fac=-0.5, fuz_fac=1.5, arm=0)
    spirals(b=-0.3, r=-disc_radius_scaled, rot_fac=-0.6, fuz_fac=1.5, arm=1)
    star_haze(scalar=9)

    model_expansion()

    # run tkinter loop
    root.mainloop()

if __name__ == '__main__':
    main()
```

A Roundabout Way to Predict Detectability

rounded_detection_practice.py

```
"""Calculate probability of detecting 32 LY-diameter radio bubble given 15.6 M
randomly distributed civilizations in the galaxy."""
import math
from random import uniform, random
from collections import Counter

# length units in light-years
DISC_RADIUS = 50000
```

```
DISC_HEIGHT = 1000
NUM_CIVS = 15600000
DETECTION_RADIUS = 16

def random_polar_coordinates_xyz():
    """Generate uniform random xyz point within a 3D disc."""
    r = random()
    theta = uniform(0, 2 * math.pi)
    x = round(math.sqrt(r) * math.cos(theta) * DISC_RADIUS, 3)
    y = round(math.sqrt(r) * math.sin(theta) * DISC_RADIUS, 3)
    z = round(uniform(0, DISC_HEIGHT), 3)
    return x, y, z

def rounded(n, base):
    """Round a number to the nearest number designated by base parameter."""
    return int(round(n/base) * base)

def distribute_civs():
    """Distribute xyz locations in galactic disc model and return list."""
    civ_locs = []
    while len(civ_locs) < NUM_CIVS:
        loc = random_polar_coordinates_xyz()
        civ_locs.append(loc)
    return civ_locs

def round_civ_locs(civ_locs):
    """Round xyz locations and return list of rounded locations."""
    # convert radius to cubic dimensions:
    detect_distance = round((4 / 3 * math.pi * DETECTION_RADIUS**3)**(1/3))
    print("\ndetection radius = {} LY".format(DETECTION_RADIUS))
    print("cubic detection distance = {} LY".format(detect_distance))

    # round civilization xyz to detection distance
    civ_locs_rounded = []

    for x, y, z in civ_locs:
        i = rounded(x, detect_distance)
        j = rounded(y, detect_distance)
```

```
            k = rounded(z, detect_distance)
            civ_locs_rounded.append((i, j, k))

    return civ_locs_rounded

def calc_prob_of_detection(civ_locs_rounded):
    """Count locations and calculate probability of duplicate values."""
    overlap_count = Counter(civ_locs_rounded)
    overlap_rollup = Counter(overlap_count.values())
    num_single_civs = overlap_rollup[1]
    prob = 1 - (num_single_civs / NUM_CIVS)

    return overlap_rollup, prob

def main():
    """Call functions and print results."""
    civ_locs = distribute_civs()
    civ_locs_rounded = round_civ_locs(civ_locs)
    overlap_rollup, detection_prob = calc_prob_of_detection(civ_locs_rounded)
    print("length pre-rounded civ_locs = {}".format(len(civ_locs)))
    print("length of rounded civ_locs_rounded = {}".format(len(civ_locs_rounded)
    print("overlap_rollup = {}\n".format(overlap_rollup))
    print("probability of detection = {0:.3f}".format(detection_prob))

    # QC step to check rounding
    print("\nFirst 3 locations pre- and post-rounding:\n")
    for i in range(3):
        print("pre-round: {}".format(civ_locs[i]))
        print("post-round: {} \n".format(civ_locs_rounded[i]))

if __name__ == '__main__':
    main()
```

Chapter 11: The Monty Hall Problem

The Birthday Paradox

birthday_paradox_practice.py

```
"""Calculate probability of a shared birthday per x number of people."""
import random

max_people = 50
num_runs = 2000

print("\nProbability of at least 2 people having the same birthday:\n")

for people in range(2, max_people + 1):
    found_shared = 0
    for run in range(num_runs):
        bdays = []
        for i in range(0, people):
            bday = random.randrange(0, 365)  # ignore leap years
            bdays.append(bday)
        set_of_bdays = set(bdays)
        if len(set_of_bdays) < len(bdays):
            found_shared += 1
    prob = found_shared/num_runs
    print("Number people = {} Prob = {:.4f}".format(people, prob))

print("""
According to the Birthday Paradox, if there are 23 people in a room,
there's a 50% chance that 2 of them will share the same birthday.
""")
```

Chapter 13: Simulating an Alien Volcano

Going the Distance

practice_45.py

```
import sys
import math
import random
import pygame as pg
```

```
pg.init()  # initialize pygame

# define color table
BLACK = (0, 0, 0)
WHITE = (255, 255, 255)
LT_GRAY = (180, 180, 180)
GRAY = (120, 120, 120)
DK_GRAY = (80, 80, 80)

class Particle(pg.sprite.Sprite):
    """Builds ejecta particles for volcano simulation."""

    gases_colors = {'SO2': LT_GRAY, 'CO2': GRAY, 'H2S': DK_GRAY, 'H2O': WHITE}

    VENT_LOCATION_XY = (320, 300)
    IO_SURFACE_Y = 308
    GRAVITY = 0.5  # pixels-per-frame
    VELOCITY_SO2 = 8  # pixels-per-frame

    # scalars (SO2 atomic weight/particle atomic weight) used for velocity
    vel_scalar = {'SO2': 1, 'CO2': 1.45, 'H2S': 1.9, 'H2O': 3.6}

    def __init__(self, screen, background):
        super().__init__()
        self.screen = screen
        self.background = background
        self.image = pg.Surface((4, 4))
        self.rect = self.image.get_rect()
        self.gas = 'SO2'
        self.color = ''
        self.vel = Particle.VELOCITY_SO2 * Particle.vel_scalar[self.gas]
        self.x, self.y = Particle.VENT_LOCATION_XY
        self.vector()

    def vector(self):
        """Calculate particle vector at launch."""
        angles = [65, 55, 45, 35, 25]  # 90 is vertical
        orient = random.choice(angles)
```

```
        if orient == 45:
            self.color = WHITE
        else:
            self.color = GRAY
        radians = math.radians(orient)
        self.dx = self.vel * math.cos(radians)
        self.dy = -self.vel * math.sin(radians)  # negative as y increases down

    def update(self):
        """Apply gravity, draw path, and handle boundary conditions."""
        self.dy += Particle.GRAVITY
        pg.draw.line(self.background, self.color, (self.x, self.y),
                     (self.x + self.dx, self.y + self.dy))
        self.x += self.dx
        self.y += self.dy
        if self.x < 0 or self.x > self.screen.get_width():
            self.kill()
        if self.y < 0 or self.y > Particle.IO_SURFACE_Y:
            self.kill()

def main():
    """Set up and run game screen and loop."""
    screen = pg.display.set_mode((639, 360))
    pg.display.set_caption("Io Volcano Simulator")
    background = pg.image.load("tvashtar_plume.gif")

    # Set up color-coded legend
    legend_font = pg.font.SysFont('None', 26)
    text = legend_font.render('White = 45 degrees', True, WHITE, BLACK)



    particles = pg.sprite.Group()

    clock = pg.time.Clock()

    while True:
        clock.tick(25)
        particles.add(Particle(screen, background))
```

```
        for event in pg.event.get():
            if event.type == pg.QUIT:
                pg.quit()
                sys.exit()

        screen.blit(background, (0, 0))
        screen.blit(text, (320, 170))

        particles.update()
        particles.draw(screen)

        pg.display.flip()

if __name__ == "__main__":
    main()
```

Chapter 16: Finding Frauds with Benford's Law

Beating Benford

beat_benford_practice.py

```
"""Manipulate vote counts so that final results conform to Benford's law."""

# example below is for Trump vs. Clinton, Illinois, 2016 Presidental Election

def load_data(filename):
    """Open a text file of numbers & turn contents into a list of integers."""
    with open(filename) as f:
        lines = f.read().strip().split('\n')
        return [int(i) for i in lines]  # turn strings to integers

def steal_votes(opponent_votes, candidate_votes, scalar):
    """Use scalar to reduce one vote count & increase another, return as lists.

    Arguments:
    opponent_votes – votes to steal from
    candidate_votes - votes to increase by stolen amount
```

```
    scalar - fractional percentage, < 1, used to reduce votes

    Returns:
    list of changed opponent votes
    list of changed candidate votes

    """
    new_opponent_votes = []
    new_candidate_votes = []
    for opp_vote, can_vote in zip(opponent_votes, candidate_votes):
        new_opp_vote = round(opp_vote * scalar)
        new_opponent_votes.append(new_opp_vote)
        stolen_votes = opp_vote - new_opp_vote
        new_can_vote = can_vote + stolen_votes
        new_candidate_votes.append(new_can_vote)
    return new_opponent_votes, new_candidate_votes

def main():
    """Run the program.

    Load data, set target winning vote count, call functions, display
    results as table, write new combined vote total as text file to
    use as input for Benford's law analysis.

    """
    # load vote data
    c_votes = load_data('Clinton_votes_Illinois.txt')
    j_votes = load_data('Johnson_votes_Illinois.txt')
    s_votes = load_data('Stein_votes_Illinois.txt')
    t_votes = load_data('Trump_votes_Illinois.txt')

    total_votes = sum(c_votes + j_votes + s_votes + t_votes)

    # assume Trump amasses a plurality of the vote with 49%
    t_target = round(total_votes * 0.49)
    print("\nTrump winning target = {:,} votes".format(t_target))

    # calculate extra votes needed for Trump victory
```

```
    extra_votes_needed = abs(t_target - sum(t_votes))
    print("extra votes needed = {:,}".format(extra_votes_needed))

    # calculate scalar needed to generate extra votes
    scalar = 1 - (extra_votes_needed / sum(c_votes + j_votes + s_votes))
    print("scalar = {:.3}".format(scalar))
    print()

    # flip vote counts based on scalar & build new combined list of votes
    fake_counts = []
    new_c_votes, new_t_votes = steal_votes(c_votes, t_votes, scalar)
    fake_counts.extend(new_c_votes)
    new_j_votes, new_t_votes = steal_votes(j_votes, new_t_votes, scalar)
    fake_counts.extend(new_j_votes)
    new_s_votes, new_t_votes = steal_votes(s_votes, new_t_votes, scalar)
    fake_counts.extend(new_s_votes)
    fake_counts.extend(new_t_votes)  # add last as has been changing up til now

    # compare old and new vote counts & totals in tabular form
    # switch-out "Trump" and "Clinton" as necessary
    for i in range(0, len(t_votes)):
        print("old Trump: {} \t new Trump: {} \t old Clinton: {} \t " \
              "new Clinton: {}".
              format(t_votes[i], new_t_votes[i], c_votes[i], new_c_votes[i]))
        print("-" * 95)
    print("TOTALS:")
    print("old Trump: {:,} \t new Trump: {:,} \t old Clinton: {:,}  " \
          "new Clinton: {:,}".format(sum(t_votes), sum(new_t_votes),
                                     sum(c_votes), sum(new_c_votes)))

    # write out a text file to use as input to benford.py program
    # this program will check conformance of faked votes to Benford's law
    with open('fake_Illinois_counts.txt', 'w') as f:
        for count in fake_counts:
            f.write("{}\n".format(count))
```

```
if __name__ == '__main__':
    main()
```